

WEEK 2 REPORT :

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Photo Frame Shopping – Product and Category Management System:

The Photo Frame Shopping system maintains information about different photo frame products and their categories. Each product belongs to a specific category. The system manages product details such as product name, price, and stock availability.

1. DESIGN PRODUCT AND CATEGORY TABLES

CATEGORY TABLE : The Category table is used to store information about different types of photo frames available in the shopping system.

```
SQL> CREATE TABLE CATEGORY (  
2     CATEGORY_ID INT PRIMARY KEY,  
3     CATEGORY_NAME VARCHAR2(50)  
4 );  
  
Table created.
```

PRODUCT TABLE: The Product table is used to store information about the photo frame products.

```
SQL> CREATE TABLE PRODUCT (  
2     PRODUCT_ID INT PRIMARY KEY,  
3     PRODUCT_NAME VARCHAR2(50),  
4     PRICE INT,  
5     STOCK INT,  
6     CATEGORY_ID INT  
7 );  
  
Table created.
```

The two tables, Category and Product, are created for managing photo frame products and their categories.

2. Define Relationships and Constraints

Relationships

One category can have many products.

The relationship between Category and Product is One to many (1-> *)

For example, the Wooden Frames category can contain different wooden photo frame products.

Therefore, the relationship between Category and Product is: One-to-Many (1 →)

Primary Key

A Primary Key is a column or field that uniquely identifies each row in a table.

In the Category table, CATEGORY_ID is the Primary Key because it uniquely identifies each category.

In the Product table, PRODUCT_ID is the Primary Key because it uniquely identifies each product.

Foreign Key

A Foreign Key is a key that links two tables.

In our Photo Frame Shopping system, CATEGORY_ID in the Product table acts as the Foreign Key.

It links the Product table with the Category table.

Therefore:

PRODUCT.CATEGORY_ID → CATEGORY.CATEGORY_ID

NOT NULL Constraint

The NOT NULL constraint ensures that a column cannot have an empty or null value.

In our system, the CATEGORY_NAME and PRODUCT_NAME should not be empty because every category and product must have a name.

CHECK Constraint

The CHECK constraint validates whether the values entered in a column satisfy a particular condition.

In our system, the price of a photo frame must always be a positive number.

Therefore, the condition used is:

PRICE > 0

UNIQUE Constraint

The UNIQUE constraint ensures that duplicate values are not allowed in a column.

In our system, the CATEGORY_NAME is unique so that two categories cannot have the same name.

PRIMARY KEY:

```
SQL> CREATE TABLE PRODUCT (  
2     PRODUCT_ID INT PRIMARY KEY,  
3     PRODUCT_NAME VARCHAR2(50),  
4     PRICE INT,  
5     STOCK INT,  
6     CATEGORY_ID INT  
7 );
```

Table created.

```
SQL> CREATE TABLE CATEGORY (  
2     CATEGORY_ID INT PRIMARY KEY,  
3     CATEGORY_NAME VARCHAR2(50)  
4 );
```

Table created.

PRODUCT_ID is a primary key and CATEGORY_ID is a primary key .There is no alter for primary key constraints.

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);
Table altered.
```

NOT NULL:

```
SQL> ALTER TABLE CATEGORY
  2  MODIFY (CATEGORY_NAME VARCHAR2(50) NOT NULL);
Table altered.
```

```
SQL> ALTER TABLE PRODUCT
  2  MODIFY (PRODUCT_NAME VARCHAR2(50) NOT NULL);
Table altered.
```

CHECK:

```
SQL> ALTER TABLE PRODUCT
  2  ADD CONSTRAINT CHK_Price
  3  CHECK (PRICE > 0);
Table altered.
```

```
SQL> ALTER TABLE PRODUCT
  2  ADD CONSTRAINT CHK_PRODUCT_STOCK
  3  CHECK (STOCK >= 0);
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);
Table altered.
```

3. Insert Values:

```

SQL> INSERT INTO CATEGORY (CATEGORY_ID, CATEGORY_NAME)
  2 VALUES (1, 'Wooden Photo Frames');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY (CATEGORY_ID, CATEGORY_NAME)
  2 VALUES (2, 'Acrylic Photo Frames');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY (CATEGORY_ID, CATEGORY_NAME)
  2 VALUES (3, 'Collage Photo Frames');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY (CATEGORY_ID, CATEGORY_NAME)
  2 VALUES (4, 'Wall Mounted Frames');

1 row created.

SQL>
SQL> INSERT INTO CATEGORY (CATEGORY_ID, CATEGORY_NAME)
  2 VALUES (5, 'Tabletop Photo Frames');

1 row created.

```

Product Table:

```

SQL> INSERT INTO PRODUCT (PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, CATEGORY_ID)
  2 VALUES (101, 'Classic Wooden Frame', 600, 20, 1);

1 row created.

SQL>
SQL> INSERT INTO PRODUCT (PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, CATEGORY_ID)
  2 VALUES (102, 'Elegant Acrylic Frame', 700, 15, 2);

1 row created.

SQL>
SQL> INSERT INTO PRODUCT (PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, CATEGORY_ID)
  2 VALUES (103, 'Family Collage Frame', 900, 10, 3);

1 row created.

SQL>
SQL> INSERT INTO PRODUCT (PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, CATEGORY_ID)
  2 VALUES (104, 'Decorative Wall Frame', 650, 12, 4);

1 row created.

SQL>
SQL> INSERT INTO PRODUCT (PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, CATEGORY_ID)
  2 VALUES (105, 'Table Photo Frame', 400, 18, 5);

1 row created.

```

Category Table with Inserted Values

```
SQL> SELECT * FROM CATEGORY;
```

```

CATEGORY_ID CATEGORY_NAME
-----
1 Wooden Photo Frames
2 Acrylic Photo Frames
3 Collage Photo Frames
4 Wall Mounted Frames
5 Tabletop Photo Frames

```

PRODUCT Table with Inserted Values

```
SQL> SELECT * FROM PRODUCT;
```

PRODUCT_ID	PRODUCT_NAME	PRICE
101	Classic Wooden Frame	600
20	1	
102	Elegant Acrylic Frame	700
15	2	
103	Family Collage Frame	900
10	3	

PRODUCT_ID	PRODUCT_NAME	PRICE
104	Decorative Wall Frame	650
12	4	
105	Table Photo Frame	400
18	5	

CRUD Operations:

Insert Record:

```
SQL> INSERT INTO PRODUCT
  2 (PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, CATEGORY_ID)
  3 VALUES (106, 'Vintage Wooden Frame', 550, 10, 1);

1 row created.
```

Update Price:

```
SQL> UPDATE PRODUCT
  2 SET PRICE = 650
  3 WHERE PRODUCT_ID = 106;

1 row updated.
```

Delete a Product:

```
SQL> DELETE FROM PRODUCT
  2 WHERE PRODUCT_ID = 106;

1 row deleted.
```

5. Generate Category-wise Product Reports Write SQL queries to: • Display all products under each category.

```
SQL> SELECT CATEGORY.CATEGORY_NAME,
2         PRODUCT.PRODUCT_NAME,
3         PRODUCT.PRICE,
4         PRODUCT.STOCK
5 FROM CATEGORY
6 JOIN PRODUCT
7 ON CATEGORY.CATEGORY_ID = PRODUCT.CATEGORY_ID;
```

CATEGORY_NAME		
PRODUCT_NAME	PRICE	STOCK
Wooden Photo Frames		
Classic Wooden Frame	600	20
Acrylic Photo Frames		
Elegant Acrylic Frame	700	15
Collage Photo Frames		
Family Collage Frame	900	10
Wall Mounted Frames		
Decorative Wall Frame	650	12
Tabletop Photo Frames		
Table Photo Frame	400	18

Count number of products in each category.

```
SQL> SELECT CATEGORY.CATEGORY_NAME,
2         COUNT(PRODUCT.PRODUCT_ID) AS PRODUCT_COUNT
3 FROM CATEGORY
4 LEFT JOIN PRODUCT
5 ON CATEGORY.CATEGORY_ID = PRODUCT.CATEGORY_ID
6 GROUP BY CATEGORY.CATEGORY_NAME;
```

CATEGORY_NAME	PRODUCT_COUNT
Acrylic Photo Frames	1
Collage Photo Frames	1
Tabletop Photo Frames	1
Wall Mounted Frames	1
Wooden Photo Frames	1

Find highest-priced product in each category.

```

SQL> SELECT CATEGORY.CATEGORY_NAME,
2         PRODUCT.PRODUCT_NAME,
3         PRODUCT.PRICE
4   FROM CATEGORY
5  JOIN PRODUCT
6  ON CATEGORY.CATEGORY_ID = PRODUCT.CATEGORY_ID
7  WHERE PRODUCT.PRICE = (
8     SELECT MAX(P2.PRICE)
9     FROM PRODUCT P2
10    WHERE P2.CATEGORY_ID = PRODUCT.CATEGORY_ID
11 );

```

CATEGORY_NAME		PRICE
PRODUCT_NAME		
Wooden Photo Frames		
Classic Wooden Frame		600
Acrylic Photo Frames		
Elegant Acrylic Frame		700
Collage Photo Frames		
Family Collage Frame		900

CATEGORY_NAME		PRICE
PRODUCT_NAME		
Wall Mounted Frames		
Decorative Wall Frame		650
Tabletop Photo Frames		
Table Photo Frame		400

Display Categories Having More Than 5 Products

```

SQL> SELECT CATEGORY.CATEGORY_NAME,
2         COUNT(PRODUCT.PRODUCT_ID) AS PRODUCT_COUNT
3   FROM CATEGORY
4  JOIN PRODUCT
5  ON CATEGORY.CATEGORY_ID = PRODUCT.CATEGORY_ID
6  GROUP BY CATEGORY.CATEGORY_NAME
7  HAVING COUNT(PRODUCT.PRODUCT_ID) > 5;

```

no rows selected

Find average product price category-wise

```
SQL> SELECT CATEGORY.CATEGORY_NAME,  
2         AVG(PRODUCT.PRICE) AS AVERAGE_PRICE  
3 FROM CATEGORY  
4 JOIN PRODUCT  
5 ON CATEGORY.CATEGORY_ID = PRODUCT.CATEGORY_ID  
6 GROUP BY CATEGORY.CATEGORY_NAME;
```

CATEGORY_NAME	AVERAGE_PRICE
Acrylic Photo Frames	700
Collage Photo Frames	900
Tabletop Photo Frames	400
Wall Mounted Frames	650
Wooden Photo Frames	600